SNARKeling Treasure Hunt Report

Version 1.0

raihanmd.xyz

April 23, 2026

SNARKeling Treasure Hunt

Raihanmd

April 23, 2026

Prepared by: Raihanmd Lead Security Researcher:

- raihanmd

Table of Contents

- Table of Contents
- Protocol Summary
  - Compatibilities
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Actors
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] Broken claimed-check allows repeated payouts for the same treasure
  - Medium
    * [M-1] Duplicate baked treasure hash reduces the effective treasure inventory to 9 and desynchronizes scoped artifacts

- Low
  * [L-1] `withdraw()` is publicly callable despite being documented as an owner withdrawal flow
  * [L-2] `Claimed` emits the caller instead of the payout recipient
- Informational
  * [I-1] Access-control and custom-error patterns are defined but not applied consistently

Protocol Summary

SNARKeling Treasure Hunt is a real-world snorkeling treasure hunt with on-chain reward claiming on an EVM blockchain. Participants physically find hidden treasures and then submit a zero-knowledge proof showing they know the correct treasure secret, without revealing the secret itself. The protocol verifies the proof on-chain and pays out an ETH reward to the designated recipient. This mechanism is built around a Noir circuit and a generated Barretenberg Honk verifier contract (more theory here https://updraft.cyfrin.io/courses/noir-programming-and-zk-circuits).

Key Features:

- Real-world treasure hunt with blockchain-based reward settlement
- ZK-SNARK based proof verification for treasure discovery
- Noir circuit that proves knowledge of a valid treasure secret without revealing it
- On-chain ETH reward distribution to a recipient bound into the proof
- Replay-resistance through recipient binding as a public input
- Owner-controlled pause/unpause, verifier update, emergency withdrawal, and post-hunt fund withdrawal flows

The protocol works as follows:

1. The organizer deploys the verifier and `TreasureHunt` contract and funds the hunt with ETH.
2. A participant finds a physical treasure associated with a unique secret string.
3. Off-chain, the participant generates a ZK proof that:
   - they know a valid treasure secret,
   - its Pedersen hash matches one of the allowed treasure hashes baked into the circuit,
   - and the proof is bound to a specific recipient address.
4. The participant submits the proof, treasure hash, and recipient to the `TreasureHunt` contract.

5. If the proof is valid and the treasure has not already been claimed, the contract transfers the fixed ETH reward to the recipient and marks the treasure as claimed.

Compatibilities

Blockchains:

- Ethereum

Protocol Assumptions:

- The hunt is preconfigured with a baked-in set of 10 valid treasure hashes in the circuit
- The contract is expected to be funded with enough ETH to cover all rewards (default deployment flow uses `100 ether`)
- The generated verifier contract must match the currently compiled circuit artifacts

Disclaimer

The raihanmd team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: aed232ce6c1ce3f909b68f1ae67978b3bfb82408

Scope

The following files are in scope for this contest:

```
contracts/
#-- scripts/
    #-- Deploy.s.sol
#-- src/
    #-- TreasureHunt.sol

circuits/
#-- src/
    #-- main.nr
```

Actors

Participant / Treasure Finder:

- Powers: Can submit a ZK proof to claim a treasure reward for a valid recipient address.
- Limitations: Cannot claim with an invalid proof, cannot claim an already-claimed treasure, cannot claim if the contract lacks sufficient funds, and cannot use invalid recipients such as the zero address, the contract address, the owner, or the caller itself.

Owner / Hunt Organizer:

- Powers: Deploys and funds the hunt, pauses/unpauses the contract, updates the verifier while paused, emergency-withdraws ETH while paused, and withdraws leftover funds after all treasures have been claimed. Owner is trusted.
- Limitations: Cannot claim treasure rewards as a participant and cannot set certain invalid recipients in emergency flows.

Executive Summary

We spent 2 hour with 1 security researcher, we are using tools foundry as a framework for testing

Issues found

Severity	Number of issues found
HIGH	1
MEDIUM	1
LOW	2
INFO	1
TOTAL	5

Findings

High

[H-1] Broken claimed-check allows repeated payouts for the same treasure

Description

- The intended claim flow is: verify a proof for `(treasureHash, recipient)`, ensure the referenced treasure has not been claimed yet, then transfer exactly one fixed reward for that treasure.
- The implementation checks `claimed[_treasureHash]` instead of `claimed[treasureHash]`. Since `_treasureHash` is never initialized in the constructor, the replay-prevention gate does not track the treasure being claimed. As a result, the same valid proof and the same `treasureHash` can be reused to receive multiple payouts.

```
function claim(bytes calldata proof, bytes32 treasureHash, address payable recipient) external nonReentrant {
    // ...
    // @> the duplicate-claim gate checks the wrong key
    if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);
    // ...
    bool ok = verifier.verify(proof, publicInputs);
    if (!ok) revert InvalidProof();

    _incrementClaimsCount();
    // @> the contract marks the correct key only after the broken check
    _markClaimed(treasureHash);
```

```
    // ...
}
```

Likelihood:

- Any user holding one valid proof can resubmit the same `(proof, treasureHash, recipient)` tuple multiple times because the duplicate-claim gate never checks the actual `treasureHash` being consumed.
- The repository's own fixtures already contain a valid proof, and the PoC shows the issue is exploitable without special setup or malformed inputs.

Impact:

- A single discovered treasure can be paid out more than once, breaking the protocol's core one-treasure/one-reward invariant.
- Repeated submissions can drain the ETH reward pool that was meant to cover all treasures.

Proof of Concept

```
function testPoCHigh_ReusedProofCanDrainRewardsForSameTreasure() public {
    (bytes memory proof, bytes32 treasureHash, address payable recipient) = _loadFixture();

    uint256 recipientBalanceBefore = recipient.balance;
    uint256 contractBalanceBefore = address(hunt).balance;

    vm.startPrank(PARTICIPANT);
    hunt.claim(proof, treasureHash, recipient);
    hunt.claim(proof, treasureHash, recipient);
    vm.stopPrank();

    assertEq(recipient.balance, recipientBalanceBefore + (2 * hunt.REWARD()));
    assertEq(address(hunt).balance, contractBalanceBefore - (2 * hunt.REWARD()));
    assertEq(hunt.claimsCount(), 2);
    assertTrue(hunt.claimed(treasureHash));
}
```

Recommended Mitigation

```
-        if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);
+        if (claimed[treasureHash]) revert AlreadyClaimed(treasureHash);
```

Medium

[M-1] Duplicate baked treasure hash reduces the effective treasure inventory to 9 and desynchronizes scoped artifacts

Description

- The protocol is documented and funded as a 10-treasure hunt: `README.md` states the circuit contains 10 valid treasure hashes, `Deploy.s.sol` documents 10 published hashes, and `TreasureHunt.sol` hardcodes `MAX_TREASURES = 10` with `REWARD = 10 ether`.
- The Noir circuit does not actually contain 10 unique treasure hashes. `circuits/src/main.nr` repeats the same baked hash in the last two slots, while `Deploy.s.sol` still documents a different 9th hash (`-441772...`) that is absent from the circuit. This desynchronizes the circuit, deployment script, and operator-facing assumptions, and leaves one documented treasure outside the actual proof-valid set.

```
// circuits/src/main.nr
global ALLOWED_TREASURE_HASHES: [Field; 10] = [
    // ...
    @> -9614350573172935800948264827865728735332357011833298311240918476355478710 92
    @> -9614350573172935800948264827865728735332357011833298311240918476355478710 92
];

// contracts/scripts/Deploy.s.sol
// @> documented hash that is missing from the circuit:
//    -4417726114039171734934559783368726413190541565291523767661452385022043124552,
```

Likelihood:

- The mismatch is already baked into the committed circuit source and mirrored by the scoped deployment script comments, so a deployment from this repository inherits the inconsistent treasure inventory by default.
- Any operator or participant relying on the published hash list in `Deploy.s.sol` can be pointed to a treasure identifier that the circuit will never accept.

Impact:

- One documented treasure becomes unclaimable through the intended ZK proof flow because its published hash is not present in the circuit's `is_allowed` set.

- Protocol lifecycle assumptions drift out of sync: the system is funded and documented as a 10-treasure hunt even though the circuit only exposes 9 unique claimable treasure identifiers.

Proof of Concept

```
function testPoCMedium_DuplicateBakedHashMismatchesDocumentedTreasureInventory() public view {
    string memory circuitSource = vm.readFile("circuits/src/main.nr");
    string memory deployScript = vm.readFile("contracts/scripts/Deploy.s.sol");
    string memory readme = vm.readFile("README.md");

    assertEq(_countOccurrences(circuitSource, DUPLICATE_CIRCUIT_HASH), 2);
    assertEq(_countOccurrences(circuitSource, MISSING_DOCUMENTED_HASH), 0);
    assertEq(_countOccurrences(deployScript, MISSING_DOCUMENTED_HASH), 1);
    assertGt(_countOccurrences(readme, "10 valid treasure hashes"), 0);
}
```

Recommended Mitigation

```
- -9614350573172935800948264827865728735332357011833298311240918476355478710 92,
- -9614350573172935800948264827865728735332357011833298311240918476355478710 92,
+ -4417726114039171734934559783368726413190541565291523767661452385022043124552,
+ -9614350573172935800948264827865728735332357011833298311240918476355478710 92,
```

Update any mirrored treasure-hash documentation in `Deploy.s.sol` and related operational materials from the same source of truth used to generate the circuit.

Low

[L-1] `withdraw()` is publicly callable despite being documented as an owner withdrawal flow

Description

- The README and NatSpec describe `withdraw()` as the owner's post-hunt withdrawal path for leftover funds.

- The implementation omits any ownership check, so any external account can trigger the withdrawal once `claimsCount >= MAX_TREASURES`. Although the funds still go to `owner`, the access-control boundary described by the project is not enforced on-chain.

```
/// @notice Allow the owner to withdraw unclaimed funds after the hunt is over.
function withdraw() external {
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");

    uint256 balance = address(this).balance;
    require(balance > 0, "NO_FUNDS_TO_WITHDRAW");
    // @> anyone can trigger the owner's withdrawal flow
    (bool sent,) = owner.call{value: balance}("");
    require(sent, "ETH_TRANSFER_FAILED");
}
```

Likelihood:

- Any arbitrary account can call `withdraw()` as soon as the treasure-count condition is satisfied and the contract later receives additional ETH.
- The PoC demonstrates the behavior with no privileged access by letting an attacker trigger the final withdrawal after the owner replenishes the contract.

Impact:

- The contract does not enforce the access-control model communicated to users and integrators.
- Unauthorized callers can force protocol settlement timing and trigger owner-only operational flows unexpectedly.

Proof of Concept

```
function testPoCLow_AnyoneCanTriggerWithdrawFlow() public {
    (bytes memory proof, bytes32 treasureHash, address payable recipient) = _loadFixture();

    _claimTenTimes(proof, treasureHash, recipient);

    uint256 ownerBalanceBeforeFund = OWNER.balance;
    vm.prank(OWNER);
    hunt.fund{value: 1 ether}();
    assertEq(OWNER.balance, ownerBalanceBeforeFund - 1 ether);

    uint256 ownerBalanceBeforeWithdraw = OWNER.balance;

    vm.prank(ATTACKER);
    hunt.withdraw();

```

```
    assertEq(address(hunt).balance, 0);
    assertEq(OWNER.balance, ownerBalanceBeforeWithdraw + 1 ether);
}
```

Recommended Mitigation

```
 function withdraw() external {
+    require(msg.sender == owner, "ONLY_OWNER");
     require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");
```

[L-2] `Claimed` emits the caller instead of the payout recipient

Description

- The `Claimed` event is the canonical on-chain record that should allow off-chain consumers to identify which treasure was claimed and who received the payout.
- The implementation emits `msg.sender` in the `recipient` slot even though the ETH transfer is sent to the `recipient` function argument. This causes the event stream to misreport the beneficiary of each successful claim.

```
event Claimed(bytes32 indexed treasureHash, address indexed recipient);

function claim(bytes calldata proof, bytes32 treasureHash, address
    payable recipient) external nonReentrant {
    // ...
    (bool sent,) = recipient.call{value: REWARD}("");
    require(sent, "ETH_TRANSFER_FAILED");

    // @> the event logs the prover/caller, not the actual reward
        recipient
    emit Claimed(treasureHash, msg.sender);
}
```

Likelihood:

- Every successful claim where the caller and payout recipient differ produces an incorrect event.
- The repository already supports this exact usage pattern by allowing a finder to bind a proof to a separate `recipient` address.

Impact:

- Indexers, analytics, and operational tooling that rely on the event stream will attribute payouts to the wrong address.

- Incident response and payout accounting become less reliable because logs diverge from actual fund flows.

Proof of Concept

```
function testPoCLow_ClaimedEventLogsCallerInsteadOfRecipient() public {
    (bytes memory proof, bytes32 treasureHash, address payable recipient) = _loadFixture();

    vm.recordLogs();

    vm.prank(PARTICIPANT);
    hunt.claim(proof, treasureHash, recipient);

    Vm.Log[] memory logs = vm.getRecordedLogs();
    bytes32 claimedTopic = keccak256("Claimed(bytes32,address)");

    bool found;
    for (uint256 i = 0; i < logs.length; i++) {
        if (logs[i].emitter == address(hunt) && logs[i].topics.length == 3 && logs[i].topics[0] == claimedTopic) {
            found = true;
            assertEq(logs[i].topics[1], bytes32(treasureHash));
            assertEq(address(uint160(uint256(logs[i].topics[2]))), PARTICIPANT);
            assertTrue(address(uint160(uint256(logs[i].topics[2]))) != recipient);
            break;
        }
    }

    assertTrue(found, "Claimed event not found");
}
```

Recommended Mitigation

```
-        emit Claimed(treasureHash, msg.sender);
+        emit Claimed(treasureHash, recipient);
```

Informational

[I-1] Access-control and custom-error patterns are defined but not applied consistently

Description

- The contract defines an `onlyOwner` modifier and a set of owner-related custom errors, which suggests the codebase intends to centralize authorization and failure handling patterns.

- In practice, most admin functions inline string-based `require(msg.sender == owner, ...)` checks instead of using the dedicated modifier, and many owner-related custom errors remain unused. This inconsistency does not currently create unauthorized access in `fund()`, `pause()`, `unpause()`, `updateVerifier()`, or `emergencyWithdraw()`, but it makes the access-control surface harder to review and easier to misuse in future changes.

```
modifier onlyOwner() {
    require(msg.sender == owner, "ONLY_OWNER");
    _;
}

error OnlyOwnerCanPause();
error OnlyOwnerCanUnpause();
error OnlyOwnerCanUpdateVerifier();
error OnlyOwnerCanEmergencyWithdraw();

function pause() external {
    // @> inline string-based auth is used instead of the shared
       modifier/error pattern
    require(msg.sender == owner, "ONLY_OWNER_CAN_PAUSE");
    paused = true;
}

function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    // @> another inline auth check instead of `onlyOwner`
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
    verifier = newVerifier;
}
```

Likelihood:

- The pattern mismatch is already present across multiple admin functions, so future edits are likely to inherit the same inconsistency.
- Reviewers must manually inspect each function body instead of relying on a single shared authorization pattern, which increases the chance of overlooking a real missing check.

Impact:

- The contract is harder to audit and maintain because access-control intent is not expressed uniformly.
- Future owner-only flows are more likely to introduce real authorization bugs by deviating from an already fragmented pattern.

Proof of Concept

```
// No executable PoC is needed for this informational issue.
```

Recommended Mitigation

```
-function pause() external {
-    require(msg.sender == owner, "ONLY_OWNER_CAN_PAUSE");
+function pause() external onlyOwner {
     paused = true;
 }

-function unpause() external {
-    require(msg.sender == owner, "ONLY_OWNER_CAN_UNPAUSE");
+function unpause() external onlyOwner {
     paused = false;
 }
```